MAULANA ABUL KALAM AZAD UNIVERSITY OF TECHNOLOGY, WEST BENGAL

Paper Code : PCC-CS501 Compiler Design

UPID : 005506

Time Allotted : 3 Hours *Full Marks :70*

The Figures in the margin indicate full marks.

Candidate are required to give their answers in their own words as far as practicable

Group-A (Very Short Answer Type Question)

1. Answer *any ten* of the following : [1 x 10 = 10]

(i) Three address code is a linearized representation of DAG. (True/False)

(ii) What is Source-To-Source Translator?

(iii) General register allocation problem is NP-Complete. (True/False)

(iv) JVM is an Interpreter. (True/False)

(v) What is the number of tokens in the C statement *printf("%d,%d",a,b);* ?

(vi) What types of grammar is used in parsing?

(vii) LEX tool is used for parsing. (True/False)

(viii) In C language type conversion can be done statically. (True/False)

(ix) Stack is used for storing local data objects. (True/False)

(x) Which type of derivation is used in bottom up parsing?

(xi) In peephole optimization the code in peephole must be contiguous. (True/False)

(xii) Number of states in SLR (1) > Number of states in CLR (1). (True/False)

Group-B (Short Answer Type Question)

Answer *any three* of the following : [5 x 3 = 15]

2. Write the differentiate between Compiler and Interpreter. [5]

3. Write the structure of Lex program. [5]

4. Why the grammar must be augmented during bottom up parsing? [5]

5. What is S-attributed and L-attributed SDD? [5]

6. Consider the grammar: T→ F T'; T' → **union** F T' | ε; F→ **not** T | **id**; Define FIRST and FOLLOW sets for the above grammar. (Assume union and not are operators) [5]

Group-C (Long Answer Type Question)

Answer *any three* of the following : [15 x 3 = 45]

7. (a) Describe all phases of 2-pass Compiler. [12]

(b) What is the purpose of Symbol Table in compiler? [3]

8. Write short notes on: [15]

i) Local Common sub-expression

ii) Dead Code and Unreachable code

iii) Reordering of three address instructions

iv) Redundant load and store

v) Machine Idioms

9. (a) Why Intermediate Code Generation is important in compiler design? [3]

(b) Explain different types of Intermediate Code Generation techniques with their advantages and disadvantages. [12]

10. (a) Lemma: Every Simple-LR (1) (SLR1) is unambiguous, but there many unambiguous grammars that are not SLR (1). [7]

Consider the grammar with productions

A → B = C | C; B → *C | id; C→ B

Find the Kernel and Non-kernel LR (0) items of each state of handle recognizing DFA of SLR (1) parsing.

(b) Build the SLR (1) parsing table and then check whether the above grammar is SLR(1) or not. [5]

(c) Check whether the above grammar is ambiguous or not. [3]

11. (a) Consider a L-attributed SDD given below: [3]

	Productions	Semantic Rules
1	E→T E'	E.node = E'.syn
		E'.inh = T.node
2	E'→ + T E_1'	E_1'.inh = new Node(+, E'.inh, T.node)
		E'.syn = E_1'.syn
3	E'→ - T E_1'	E_1'.inh = new Node(-, E'.inh, T.node)
		E'.syn = E_1'.syn
4	E'→ ε	E'.syn = E'.inh
5	T → (E)	T.node = E.node
6	T → id	T.node = new Leaf(id, id.entry)
7	T → num	T.node = new Leaf(num, num.val)

Where, syn and node are synthesized attributes and inh is an inherited attributes. Here E, E1 both are same, and for differentiate between left E and right E, we use E1 in right.

Draw the parse tree for the input **a - 4 + c**.

(b) Build the dependency DAG for evaluation order of attributes of input a - 4 + c and then draw the Syntax tree for the expression over this DAG. [12]

*** END OF PAPER ***